LISTA DE EXERCÍCIOS — ALGORITMOS DE BUSCA EM PYTHON

Busca Sequencial e Busca Binária

Orientações: Desenvolva as soluções em Python. Sempre que solicitado, implemente o algoritmo manualmente, sem utilizar métodos prontos como *index()* ou operadores que realizem diretamente a busca.

PARTE 1 — BUSCA SEQUENCIAL

1. Crie uma lista com 10 números inteiros e desenvolva um programa que solicite ao usuário um número. Utilize a busca sequencial para informar se o número está presente na lista.

2. Dada uma lista de nomes, solicite um nome ao usuário e utilize a busca sequencial para informar a posição em que ele foi encontrado. Caso não exista, informe uma mensagem adequada.

3. Crie uma função **busca_sequencial(lista, valor)** que retorne o índice do valor procurado ou **-1** caso ele não esteja presente.

4. Dada uma lista de notas de alunos, solicite uma nota e verifique, por meio da busca sequencial, se ela aparece na lista. Informe também quantas vezes ela ocorre.

5. Crie uma lista de produtos e uma lista correspondente de preços. Solicite o nome de um produto e utilize busca sequencial para localizar o produto e exibir seu preço.

6. Dada uma lista de números inteiros, utilize busca sequencial para localizar o maior valor sem utilizar as funções *max()* ou *sort()*.

7. Crie uma lista com números aleatórios. Solicite um valor e, usando busca sequencial, informe todas as posições em que esse valor aparece.

8. Desenvolva uma função que receba uma lista e um número e retorne **True** se o número existir na lista e **False** caso contrário. A função deve utilizar busca sequencial.

9. Uma escola possui uma lista com os números de matrícula dos alunos. Solicite uma matrícula e verifique se o aluno está cadastrado utilizando busca sequencial.

10. Dada uma lista de palavras, utilize busca sequencial para encontrar a palavra com maior quantidade de caracteres. Não utilize *max()* com chave.

11. Crie um programa que possua uma lista de CPFs fictícios. Solicite um CPF e informe se ele está cadastrado. Conte também quantas comparações foram necessárias até encontrá-lo.

12. Implemente uma busca sequencial que pare assim que encontrar o valor procurado. Exiba o valor encontrado, sua posição e o número de comparações realizadas.

PARTE 2 — BUSCA BINÁRIA

13. Crie uma lista ordenada de números inteiros e implemente manualmente a busca binária para localizar um número informado pelo usuário.

14. Crie uma função **busca_binaria(lista, valor)** que receba uma lista ordenada e retorne o índice do valor procurado ou **-1** caso ele não seja encontrado.

15. Dada uma lista ordenada de nomes, implemente uma busca binária para verificar se um nome informado pelo usuário está presente na lista.

16. Crie uma lista ordenada com 20 números. Realize uma busca binária e exiba, a cada iteração, os valores das variáveis **inicio**, **fim** e **meio**.

17. Dada uma lista ordenada de códigos de produtos, solicite um código e utilize busca binária para verificar se ele está cadastrado.

18. Modifique a busca binária para contar quantas comparações são necessárias para localizar um elemento.

19. Implemente uma busca binária que retorne **True** ou **False**, sem retornar o índice do elemento.

20. Dada uma lista ordenada contendo números pares de 2 até 200, utilize busca binária para localizar um número informado pelo usuário.

21. Crie uma lista ordenada com valores repetidos. Implemente uma busca binária capaz de localizar uma ocorrência do valor procurado e informe seu índice.

22. Desenvolva uma versão da busca binária que encontre a **primeira ocorrência** de um valor repetido em uma lista ordenada.

23. Desenvolva uma versão da busca binária que encontre a **última ocorrência** de um valor repetido em uma lista ordenada.

24. Dada uma lista ordenada de idades, utilize busca binária para verificar se uma idade específica está presente e informe quantos elementos foram descartados durante o processo.

PARTE 3 — PROBLEMAS APLICADOS E COMPARAÇÃO

25. **Sistema de Biblioteca:** Uma biblioteca possui uma lista ordenada de códigos de livros. Implemente uma busca binária para verificar se um código informado está disponível. Caso esteja, informe a posição do livro na lista.

26. **Controle de Estoque:** Uma loja possui uma lista de produtos e deseja localizar um produto pelo código. Implemente uma solução com busca sequencial e outra com busca binária. Compare os resultados.

27. **Cadastro de Alunos:** Dada uma lista de matrículas, crie um programa que permita ao usuário escolher entre busca sequencial ou busca binária. Para a busca binária, garanta que a lista esteja ordenada.

28. **Comparação de Desempenho:** Crie uma lista ordenada com 1.000 números. Procure o último elemento usando busca sequencial e busca binária. Conte o número de comparações realizadas em cada algoritmo e apresente uma conclusão.

29. **Agenda Telefônica:** Crie uma lista de contatos ordenada alfabeticamente. O usuário deve informar um nome e o programa deverá localizá-lo usando busca binária. Se encontrado, exiba também o telefone correspondente.

30. **Desafio:** Crie um menu com as opções: (1) cadastrar valores, (2) exibir lista, (3) ordenar lista, (4) realizar busca sequencial, (5) realizar busca binária e (0) sair. A busca binária só poderá ser realizada quando a lista estiver ordenada.

> **Desafio de reflexão:** Explique, com suas próprias palavras, em quais situações a busca sequencial é mais adequada e em quais situações a busca binária apresenta vantagem. Considere também a necessidade de a lista estar ordenada.